2021 IEEE/CVF International Conference on Computer Vision (ICCV) | 978-1-6654-2812-5/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICCV48922.2021.00951

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron[1,2] Hugo Touvron[1,3] Ishan Misra[1] Hervé Jegou[1]
Julien Mairal[2] Piotr Bojanowski[1] Armand Joulin[1]

[1] Facebook AI Research [2] Inria* [3] Sorbonne University

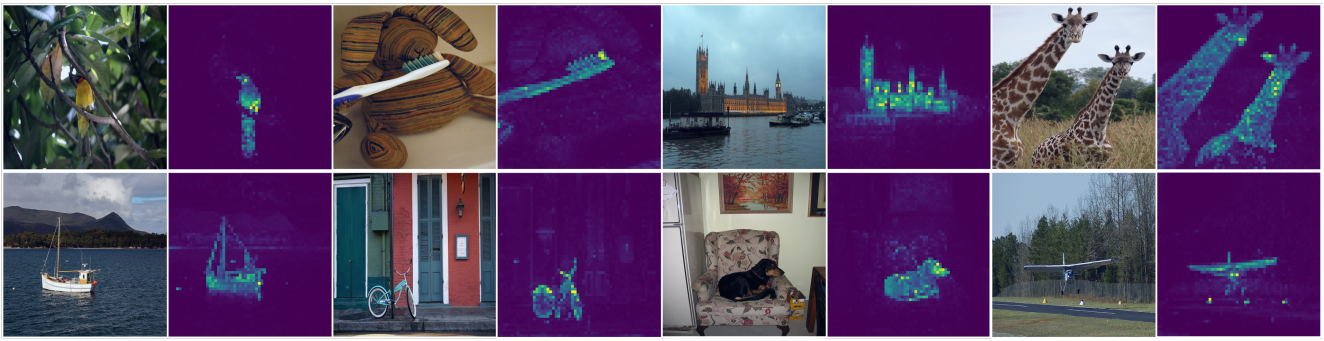


Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the `[CLS]` token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

Abstract

*In this paper, we question if self-supervised learning provides new properties to Vision Transformer (ViT) [16] that stand out compared to convolutional networks (convnets). Beyond the fact that adapting self-supervised methods to this architecture works particularly well, we make the following observations: first, self-supervised ViT features contain explicit information about the semantic segmentation of an image, which does not emerge as clearly with supervised ViTs, nor with convnets. Second, these features are also excellent k-NN classifiers, reaching 78.3% top-1 on ImageNet with a small ViT. Our study also underlines the importance of momentum encoder [26], multi-crop training [9], and the use of small patches with ViTs. We implement our findings into a simple self-supervised method, called DINO, which we interpret as a form of self-**di**stillation with **no** labels. We show the synergy between DINO and ViTs by achieving 80.1% top-1 on ImageNet in linear evaluation with ViT-Base.*

*Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, 38000 Grenoble, France.
Correspondence: mathilde@fb.com
Code: https://github.com/facebookresearch/dino

1. Introduction

Transformers [57] have recently emerged as an alternative to convolutional neural networks (convnets) for visual recognition [16, 56, 68]. Their adoption has been coupled with a training strategy inspired by natural language processing (NLP), that is, pretraining on large quantities of data and finetuning on the target dataset [15, 45]. The resulting Vision Transformers (ViT) [16] are competitive with convnets but, they have not yet delivered clear benefits over them: they are computationally more demanding, require more training data, and their features do not exhibit unique properties.

In this paper, we question whether the muted success of Transformers in vision can be explained by the use of supervision in their pretraining. Our motivation is that one of the main ingredients for the success of Transformers in NLP was the use of self-supervised pretraining, in the form of close procedure in BERT [15] or language modeling in GPT [45]. These self-supervised pretraining objectives use the words in a sentence to create pretext tasks that provide a richer learning signal than the supervised objective of predicting a single label per sentence. Similarly, in images, image-level supervision often reduces the rich visual information contained in an image to a single concept selected from a predefined set of a few thousand categories of objects [49].

While the self-supervised pretext tasks used in NLP are

978-1-6654-2812-5/21/$31.00 ©2021 IEEE
DOI 10.1109/ICCV48922.2021.00951

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

text specific, many existing self-supervised methods have shown their potential on images with convnets [9, 11, 23, 26]. They typically share a similar structure but with different components designed to avoid trivial solutions (collapse) or to improve performance [14]. In this work, inspired from these methods, we study the impact of self-supervised pretraining on ViT features. Of particular interest, we have identified several interesting properties that do not emerge with supervised ViTs, nor with convnets:

- Self-supervised ViT features explicitly contain the scene layout and, in particular, object boundaries, as shown in Figure 1. This information is directly accessible in the self-attention modules of the last block.
- Self-supervised ViT features perform particularly well with a basic nearest neighbors classifier (k-NN) *without any finetuning, linear classifier nor data augmentation*, achieving 78.3% top-1 accuracy on ImageNet.

The emergence of segmentation masks seems to be a property shared across self-supervised methods. However, the good performance with k-NN only emerge when combining certain components such as momentum encoder [26] and multi-crop augmentation [9]. Another finding from our study is the importance of using smaller patches with ViTs to improve the quality of the resulting features.

Overall, our findings about the importance of these components lead us to design a simple self-supervised approach that can be interpreted as a form of knowledge **di**stillation [28] with **no** labels. The resulting framework, DINO, simplifies self-supervised training by directly predicting the output of a teacher network—built with a momentum encoder—by using a standard cross-entropy loss. Interestingly, our method can work with only a centering and sharpening of the teacher output to avoid collapse, while other popular components such as predictor [23], advanced normalization [9] or contrastive loss [26] add little benefits in terms of stability or performance. Of particular importance, our framework is flexible and works on both convnets and ViTs without the need to modify the architecture, nor adapt internal normalizations [47].

We further validate the synergy between DINO and ViT by outperforming previous self-supervised features on the ImageNet linear classification benchmark with 80.1% top-1 accuracy with a ViT-Base with small patches. We also confirm that DINO works with convnets by matching the state of the art with a ResNet-50 architecture. Finally, we discuss different scenarios to use DINO with ViTs in case of limited computation and memory capacity. In particular, training DINO with ViT takes just two 8-GPU servers over 3 days to achieve 76.1% on ImageNet linear benchmark, which outperforms self-supervised systems based on convnets of comparable sizes with significantly reduced compute requirements [9, 23].

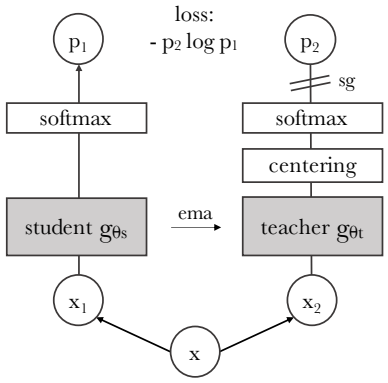


Figure 2: **Self-distillation with no labels.** We illustrate DINO in the case of one single pair of views (x_1, x_2) for simplicity. The model passes two different random transformations of an input image to the student and teacher networks. Both networks have the same architecture but different parameters. The output of the teacher network is centered with a mean computed over the batch. Each networks outputs a K dimensional feature that is normalized with a temperature softmax over the feature dimension. Their similarity is then measured with a cross-entropy loss. We apply a stop-gradient (sg) operator on the teacher to propagate gradients only through the student. The teacher parameters are updated with an exponential moving average (ema) of the student parameters.

2. Related work

Self-supervised learning. A large body of work on self-supervised learning focuses on discriminative approaches coined *instance classification* [11, 17, 26, 60], which considers each image a different class and trains the model by discriminating them up to data augmentations. However, explicitly learning a classifier to discriminate between all images [17] does not scale well with the number of images. Wu *et al*. [60] propose to use a noise contrastive estimator (NCE) [25] to compare instances instead of classifying them. A caveat of this approach is that it requires comparing features from a large number of images simultaneously. In practice, this requires large batches [11] or memory banks [26, 60]. Several variants allow automatic grouping of instances in the form of clustering [2, 7, 8, 21, 29, 35, 61, 65, 69].

Recent works have shown that we can learn unsupervised features without discriminating between images. Of particular interest, Grill *et al*. [23] propose a metric-learning formulation called BYOL, where features are trained by matching them to representations obtained with a momentum encoder. It has been shown that methods like BYOL work even without a momentum encoder, at the cost of a drop of performance [14, 23]. Several other works echo this direction, showing that one can train features matching them to a uniform distribution on the ℓ_2 hypersphere [5] or by using whitening [19, 66]. Our approach takes its inspiration from BYOL but operates with a different similarity matching

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

loss and uses the exact same architecture for the student and the teacher. That way, our work completes the interpretation initiated in BYOL of self-supervised learning as a form of Mean Teacher self-distillation [52] with no labels.

Self-training and knowledge distillation. Self-training aims at improving the quality of features by propagating a small initial set of annotations to a large set of unlabeled instances. This propagation can either be done with hard assignments of labels [34, 63, 64] or with a soft assignment [62]. When using soft labels, the approach is often referred to as knowledge distillation [6, 28] and has been primarily designed to train a small network to mimic the output of a larger network to compress models. Xie *et al*. [62] have recently shown that distillation could be used to propagate soft pseudo-labels to unlabelled data in a self-training pipeline, drawing an essential connection between self-training and knowledge distillation. Our work builds on this relation and extends knowledge distillation to the case where no labels are available. Previous works have also combined self-supervised learning and knowledge distillation, enabling self-supervised model compression [20] and performance gains [12, 38]. However, these works rely on a *pre-trained* fixed teacher while our teacher is dynamically built during training. This way, knowledge distillation, instead of being used as a post-processing step to self-supervised pre-training, is directly cast as a self-supervised objective. Finally, our work is also related to *codistillation* [1] where student and teacher have the same architecture and use distillation during training. However, the teacher in *codistillation* is also distilling from the student, while it is updated with a momentum average of the student in our work.

3. Approach

3.1. SSL with Knowledge Distillation

The framework used for this work, DINO, shares the same overall structure as recent self-supervised approaches [9, 14, 11, 23, 26]. However, our method shares also similarities with knowledge distillation [28] and we present it under this angle. We illustrate DINO in Figure 2 and propose a pseudo-code implementation in Algorithm 1.

Knowledge distillation is a learning paradigm where we train a student network g_{θ_s} to match the output of a given teacher network g_{θ_t}, parameterized by θ_s and θ_t respectively. Given an input image x, both networks output probability distributions over K dimensions denoted by P_s and P_t. The probability P is obtained by normalizing the output of the network g with a softmax function. More precisely,

$$P_s(x)^{(i)} = \frac{\exp(g_{\theta_s}(x)^{(i)}/\tau_s)}{\sum_{k=1}^{K} \exp(g_{\theta_s}(x)^{(k)}/\tau_s)}, \qquad (1)$$

with $\tau_s > 0$ a temperature parameter that controls the sharpness of the output distribution, and a similar formula holds for P_t with temperature τ_t. Given a fixed teacher network g_{θ_t}, we learn to match these distributions by minimizing the cross-entropy loss w.r.t. the parameters of the student network θ_s:

$$\min_{\theta_s} H(P_t(x), P_s(x)), \qquad (2)$$

where $H(a, b) = -a \log b$.

In the following, we detail how we adapt the problem in Eq. (2) to self-supervised learning. First, we construct different distorted views, or crops, of an image with multi-crop strategy [9]. More precisely, from a given image, we generate a set V of different views. This set contains two *global* views, x_1^g and x_2^g and several *local* views of smaller resolution. All crops are passed through the student while only the *global* views are passed through the teacher, therefore encouraging "local-to-global" correspondences. We minimize the loss:

$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x), P_s(x')). \qquad (3)$$

This loss is general and can be used on any number of views, even only 2. However, we follow the standard setting for multi-crop by using 2 global views at resolution 224^2 covering a large (for example greater than 50%) area of the original image, and several local views of resolution 96^2 covering only small areas (for example less than 50%) of the original image. We refer to this setting as the basic parametrization of DINO, unless mentioned otherwise.

Both networks share the same architecture g with different sets of parameters θ_s and θ_t. We learn the parameters θ_s by minimizing Eq. (3) with stochastic gradient descent.

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```


Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

Table 1: **Networks configuration.** "Blocks" is the number of Transformer blocks, "dim" is channel dimension and "heads" is the number of heads in multi-head attention. "# tokens" is the length of the token sequence when considering 224^2 resolution inputs, "# params" is the total number of parameters (without counting the projection head) and "im/s" is the inference time on a NVIDIA V100 GPU with 128 samples per forward.

model	blocks	dim	heads	#tokens	#params	im/s
ResNet-50	–	2048	–	–	23M	1237
ViT-S/16	12	384	6	197	21M	1007
ViT-S/8	12	384	6	785	21M	180
ViT-B/16	12	768	12	197	85M	312
ViT-B/8	12	768	12	785	85M	63

Teacher network. Unlike knowledge distillation, we do not have a teacher g_{θ_t} given *a priori* and hence, we build it from past iterations of the student network. We study different update rules for the teacher in Appendix and show that freezing the teacher network over an epoch works surprisingly well in our framework, while copying the student weight for the teacher fails to converge. Of particular interest, using an exponential moving average (EMA) on the student weights, i.e., a momentum encoder [26], is particularly well suited for our framework. The update rule is $\theta_t \leftarrow \lambda\theta_t + (1-\lambda)\theta_s$, with λ following a cosine schedule from 0.996 to 1 during training [23]. Originally the momentum encoder has been introduced as a substitute for a queue in contrastive learning [26]. However, in our framework, its role differs since we do not have a queue nor a contrastive loss, and may be closer to the role of the mean teacher used in self-training [52]. Indeed, we observe that this teacher performs a form of model ensembling similar to Polyak-Ruppert averaging with an exponential decay [41, 48]. Using Polyak-Ruppert averaging for model ensembling is a standard practice to improve the performance of a model [31]. We observe that this teacher has better performance than the student throughout the training, and hence, guides the training of the student by providing target features of higher quality. This dynamic was not observed in previous works [23, 47].

Network architecture. The neural network g is composed of a backbone f (ViT [16] or ResNet [27]), and of a projection head h: $g = h \circ f$. The features used in downstream tasks are the backbone f output. The projection head consists of a 3-layer multi-layer perceptron (MLP) with hidden dimension 2048 followed by ℓ_2 normalization and a weight normalized fully connected layer [50] with K dimensions, which is similar to the design from SwAV [9]. We have tested other projection heads and this particular design appears to work best for DINO (see Appendix). We do not use a predictor [23, 14], resulting in the exact same architecture in both student and teacher networks. Of particular interest, we note that unlike standard convnets, ViT architectures do not use batch normalizations (BN) by default. Therefore, when applying DINO to ViT we do not use any BN also in the projection heads, making the system *entirely BN-free*.

Avoiding collapse. Several self-supervised methods differ by the operation used to avoid collapse, either through contrastive loss [60], clustering constraints [7, 9], predictor [23] or batch normalizations [23, 47]. While our framework can be stabilized with multiple normalizations [9], it can also work with only a centering and sharpening of the momentum teacher outputs to avoid model collapse. As shown experimentally in Appendix, centering prevents one dimension to dominate but encourages collapse to the uniform distribution, while the sharpening has the opposite effect. Applying both operations balances their effects which is sufficient to avoid collapse in presence of a momentum teacher. Choosing this method to avoid collapse trades stability for less dependence over the batch: the centering operation only depends on first-order batch statistics and can be interpreted as adding a bias term c to the teacher: $g_t(x) \leftarrow g_t(x) + c$. The center c is updated with an exponential moving average, which allows the approach to work well across different batch sizes as shown in Appendix.

$$c \leftarrow mc + (1-m)\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i), \tag{4}$$

where $m > 0$ is a rate parameter and B is the batch size. Output sharpening is obtained by using a low value for the temperature τ_t in the teacher softmax normalization.

3.2. Implementation and evaluation protocols

In this section, we provide the implementation details to train with DINO and present the evaluation protocols used in our experiments.

Vision Transformer. We briefly describe the mechanism of the Vision Transformer (ViT) [16, 57] and refer to Vaswani *et al*. [57] for details about Transformers and to Dosovitskiy *et al*. [16] for its adaptation to images. We follow the implementation used in `DeiT` [56]. We summarize the configuration of the different networks used in this paper in Table 1. The ViT architecture takes as input a grid of non-overlapping contiguous image patches of resolution $N \times N$. In this paper we typically use $N = 16$ ("/16") or $N = 8$ ("/8"). The patches are then passed through a linear layer to form a set of embeddings. We add an extra learnable token to the sequence [15, 16]. The role of this token is to aggregate information from the entire sequence and we attach the projection head h at its output. We refer to this token as the class token `[CLS]` for consistency with

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

previous works[15, 16, 56], even though it is not attached to any label nor supervision in our case. The set of patch tokens and `[CLS]` token are fed to a standard Transformer network with a "pre-norm" layer normalization [10, 32]. The Transformer is a sequence of self-attention and feed-forward layers, paralleled with skip connections. The self-attention layers update the token representations by looking at the other token representations with an attention mechanism [3].

Implementation details. We pretrain the models on the ImageNet dataset [49] without labels. We train with the adamw optimizer [37] and a batch size of 1024, distributed over 16 GPUs when using ViT-S/16. The learning rate is linearly ramped up during the first 10 epochs to its base value determined with the following linear scaling rule [22]: $lr = 0.0005 * \text{batchsize}/256$. After this warmup, we decay the learning rate with a cosine schedule [36]. The weight decay also follows a cosine schedule from 0.04 to 0.4. The temperature τ_s is set to 0.1 while we use a linear warm-up for τ_t from 0.04 to 0.07 during the first 30 epochs. We follow the data augmentations of BYOL [23] (color jittering, Gaussian blur and solarization) and multi-crop [9] with a bicubic interpolation to adapt the position embeddings to the scales [16, 56]. The code and models to reproduce our results is publicly available at https://github.com/facebookresearch/dino.

Evaluation protocols. Standard protocols for self-supervised learning are to either learn a linear classifier on frozen features [67, 26] or to finetune the features on downstream tasks. For linear evaluations, we apply random resize crops and horizontal flips augmentation during training, and report accuracy on a central crop. For finetuning evaluations, we initialize networks with the pretrained weights and adapt them during training. However, both evaluations are sensitive to hyperparameters, and we observe a large variance in accuracy between runs when varying the learning rate for example. We thus also evaluate the quality of features with a simple weighted nearest neighbor classifier (k-NN) as in [60]. We freeze the pretrain model to compute and store the features of the training data of the downstream task. The nearest neighbor classifier then matches the feature of an image to the k nearest stored features that votes for the label. We sweep over different number of nearest neighbors and find that 20 NN is consistently working the best for most of our runs. This evaluation protocol does not require any other hyperparameter tuning, nor data augmentation and can be run with only one pass over the downstream dataset, greatly simplifying the feature evaluation.

Table 2: **Linear and k-NN classification on ImageNet.** We report top-1 accuracy for linear and k-NN evaluations on the validation set of ImageNet for different self-supervised methods. We focus on ResNet-50 and ViT-small architectures, but also report the best results obtained across architectures. * are run by us. We run the k-NN evaluation for models with official released weights. The throughput (im/s) is calculated on a NVIDIA V100 GPU with 128 samples per forward. Parameters (M) are of the feature extractor.

Method	Arch.	Param.	im/s	Linear	k-NN
Supervised	RN50	23	1237	79.3	79.3
SCLR [11]	RN50	23	1237	69.1	60.7
MoCov2 [13]	RN50	23	1237	71.1	61.9
InfoMin [54]	RN50	23	1237	73.0	65.3
BarlowT [66]	RN50	23	1237	73.2	66.0
OBoW [21]	RN50	23	1237	73.8	61.9
BYOL [23]	RN50	23	1237	74.4	64.8
DCv2 [9]	RN50	23	1237	75.2	67.1
SwAV [9]	RN50	23	1237	**75.3**	65.7
DINO	RN50	23	1237	**75.3**	**67.5**
Supervised	ViT-S	21	1007	79.8	79.8
BYOL* [23]	ViT-S	21	1007	71.4	66.6
MoCov2* [13]	ViT-S	21	1007	72.7	64.4
SwAV* [9]	ViT-S	21	1007	73.5	66.3
DINO	ViT-S	21	1007	**77.0**	**74.5**
Comparison across architectures					
SCLR [11]	RN50w4	375	117	76.8	69.3
SwAV [9]	RN50w2	93	384	77.3	67.3
BYOL [23]	RN50w2	93	384	77.4	–
DINO	ViT-B/16	85	312	78.2	76.1
SwAV [9]	RN50w5	586	76	78.5	67.1
BYOL [23]	RN50w4	375	117	78.6	–
BYOL [23]	RN200w2	250	123	79.6	73.9
DINO	ViT-S/8	21	180	79.7	**78.3**
SCLRv2 [12]	RN152w3+SK	794	46	79.8	73.1
DINO	ViT-B/8	85	63	**80.1**	77.4

4. Main Results

We first validate the DINO framework used in this study with the standard self-supervised benchmark on ImageNet. We then study the properties of the resulting features for retrieval, object discovery and transfer-learning.

4.1. Comparing with SSL frameworks on ImageNet

We consider two different settings: comparison with the same architecture and across architectures.

Comparing with the same architecture. In top panel of Table 2, we compare DINO with other self-supervised methods with the same architecture, either a ResNet-50 [27] or a ViT-small (ViT-S) [56]. The choice of ViT-S is motivated by its similarity with ResNet-50 along several axes: number of parameters (21M vs 23M), throughput (1237/sec VS

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

Table 3: **Image retrieval.** We compare the performance in retrieval of off-the-shelf features pretrained with supervision or with DINO on ImageNet and Google Landmarks v2 (GLDv2) dataset. We report mAP on revisited Oxford and Paris. Pretraining with DINO on a landmark dataset performs particularly well. For reference, we also report the best retrieval method with off-the-shelf features [46].

			$\mathcal{R}$Ox		$\mathcal{R}$Par	
Pretrain	Arch.	Pretrain	M	H	M	H
Sup. [46]	RN101+R-MAC	ImNet	49.8	18.5	74.0	**52.1**
Sup.	ViT-S/16	ImNet	33.5	8.9	63.0	37.2
DINO	ResNet-50	ImNet	35.4	11.1	55.9	27.5
DINO	ViT-S/16	ImNet	41.8	13.7	63.1	34.4
DINO	ViT-S/16	GLDv2	**51.5**	**24.3**	**75.3**	51.6

1007 im/sec) and supervised performance on ImageNet with the training procedure of [56] (79.3% VS 79.8%). We explore variants of ViT-S in Appendix. First, we observe that DINO performs on par with the state of the art on ResNet-50, validating that DINO works in the standard setting. When we switch to a ViT architecture, DINO outperforms BYOL, MoCov2 and SwAV by +3.5% with linear classification and by +7.9% with k-NN evaluation. More surprisingly, the performance with a simple k-NN classifier is almost on par with a linear classifier (74.5% versus 77.0%). This property emerges only when using DINO with ViT architectures, and does not appear with other existing self-supervised methods nor with a ResNet-50.

Comparing across architectures. On the bottom panel of Table 2, we compare the best performance obtained across architectures. The interest of this setting is not to compare methods directly, but to evaluate the limits of a ViT trained with DINO when moving to larger architectures. While training a larger ViT with DINO improves the performance, reducing the size of the patches ("/8" variants) has a bigger impact on the performance. While reducing the patch size do not add parameters, it still leads to a significant reduction of running time, and larger memory usage. Nonetheless, a base ViT with 8×8 patches trained with DINO achieves 80.1% top-1 in linear classification and 77.4% with a k-NN classifier with $10\times$ less parameters and $1.4\times$ faster run time than previous state of the art [12].

4.2. Properties of ViT trained with SSL

We evaluate properties of the DINO features in terms of nearest neighbor search, retaining information about object location and transferability to downstream tasks.

Table 4: **Copy detection.** We report the mAP performance in copy detection on Copydays "strong" subset [18]. For reference, we also report the performance of the multigrain model [4], trained specifically for particular object retrieval.

Method	Arch.	Dim.	Resolution	mAP
Multigrain [4]	ResNet-50	2048	224^2	75.1
Multigrain [4]	ResNet-50	2048	largest side 800	82.5
Supervised [56]	ViT-B/16	1536	224^2	76.4
DINO	ViT-B/16	1536	224^2	81.7
DINO	ViT-B/8	1536	320^2	**85.5**

4.2.1 Nearest neighbor retrieval with DINO ViT

The results on ImageNet classification have exposed the potential of our features for tasks relying on nearest neighbor retrieval. In this set of experiments, we further consolidate this finding on landmark retrieval and copy detection tasks.

Image Retrieval. We consider the revisited [43] Oxford and Paris image retrieval datasets [40]. They contain 3 different splits of gradual difficulty with query/database pairs. We report the Mean Average Precision (mAP) for the Medium (M) and Hard (H) splits. In Table 3, we compare the performance of different *off-the-shelf* features obtained with either supervised or DINO training. We freeze the features and directly apply k-NN for retrieval. We observe that DINO features outperform those trained on ImageNet with labels.

An advantage of SSL approaches is that they can be trained on any dataset, without requiring any form of annotations. We train DINO on the 1.2M clean set from Google Landmarks v2 (GLDv2) [59], a dataset of landmarks designed for retrieval purposes. DINO ViT features trained on GLDv2 are remarkably good, outperforming previously published methods based on off-the-shelf descriptors [55, 46].

Copy detection. We also evaluate the performance of ViTs trained with DINO on a copy detection task. We report the mean average precision on the "strong" subset of the INRIA Copydays dataset [18]. The task is to recognize images that have been distorted by blur, insertions, print and scan, etc. Following prior work [4], we add 10k distractor images randomly sampled from the YFCC100M dataset [53]. We perform copy detection directly with cosine similarity on the features obtained from our pretrained network. The features are obtained as the concatenation of the output `[CLS]` token and of the GeM pooled [44] output patch tokens. This results in a 1536d descriptor for ViT-B. Following [4], we apply whitening on the features. We learn this transformation on an extra 20K random images from YFCC100M, distincts from the distractors. Table 4 shows that ViT trained with DINO is very competitive on copy detection.

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

Table 5: **DAVIS 2017 Video object segmentation.** We evaluate the quality of frozen features on video instance tracking. We report mean region similarity $\mathcal{J}_m$ and mean contour-based accuracy $\mathcal{F}_m$. We compare with existing self-supervised methods and a supervised ViT-S/8 trained on ImageNet. Image resolution is 480p.

Method	Data	Arch.	$(\mathcal{J}\&\mathcal{F})_m$	$\mathcal{J}_m$	$\mathcal{F}_m$
Supervised					
ImageNet	INet	ViT-S/8	66.0	63.9	68.1
STM [39]	I/D/Y	RN50	81.8	79.2	84.3
Self-supervised					
CT [58]	VLOG	RN50	48.7	46.4	50.0
MAST [33]	YT-VOS	RN18	65.5	63.3	67.6
STC [30]	Kinetics	RN18	67.6	64.8	70.2
DINO	INet	ViT-S/16	61.8	60.2	63.4
DINO	INet	ViT-B/16	62.3	60.7	63.9
DINO	INet	ViT-S/8	**69.9**	**66.6**	**73.1**
DINO	INet	ViT-B/8	**71.4**	**67.9**	**74.9**

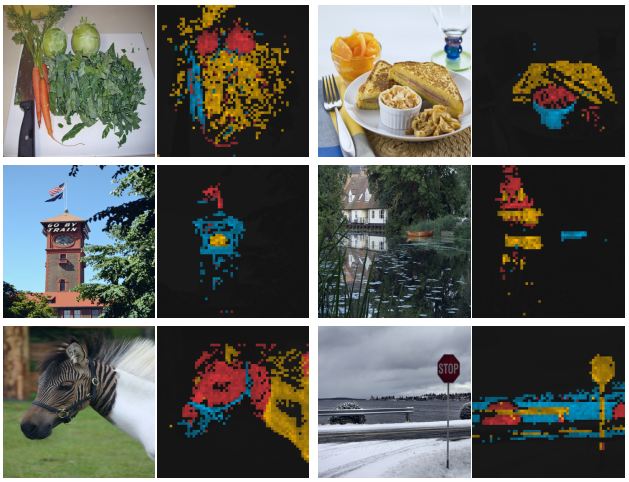


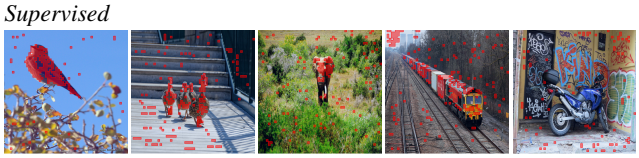
Figure 3: **Attention maps from multiple heads.** We consider the heads from the last layer of a ViT-S/8 trained with DINO and display the self-attention for `[CLS]` token query. Different heads, materialized by different colors, focus on different locations that represents different objects or parts (more examples in Appendix).

4.2.2 Discovering the semantic layout of scenes

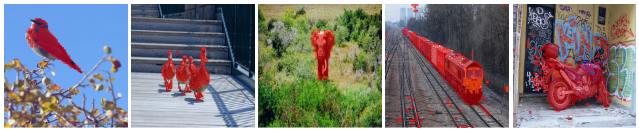
As shown qualitatively in Figure 1, our self-attention maps contain information about the segmentation of an image. In this study, we measure this property on a standard benchmark as well as by directly probing the quality of masks generated from these attention maps.

Video instance segmentation. In Tab. 5, we evaluate the output patch tokens on the DAVIS-2017 video instance segmentation benchmark [42]. We follow the experimental protocol in Jabri *et al*. [30] and segment scenes with a nearest-neighbor between consecutive frames; we thus do not train any model on top of the features, nor finetune any weights for the task. We observe in Tab. 5 that even though our training objective nor our architecture are designed for dense tasks, the performance is competitive on this benchmark. Since the network is not finetuned, the output of the model must have retained some spatial information. Finally, for this dense recognition task, the variants with small patches ("/8") perform much better (+9.1% $(\mathcal{J}\&\mathcal{F})_m$ for ViT-B).

Supervised



DINO



	Random	Supervised	DINO
ViT-S/16	22.0	27.3	45.9
ViT-S/8	21.8	23.7	44.7

Figure 4: **Segmentations from supervised versus DINO.** We visualize masks obtained by thresholding the self-attention maps to keep 60% of the mass. On top, we show the resulting masks for a ViT-S/8 trained with supervision and DINO. We show the best head for both models. The table at the bottom compares the Jaccard similarity between the ground truth and these masks on the validation images of PASCAL VOC12 dataset.

Probing the self-attention map. In Fig. 3, we show that different heads can attend to different semantic regions of an image, even when they are occluded (the bushes on the third row) or small (the flag on the second row). Visualizations are obtained with 480p images, resulting in sequences of 3601 tokens for ViT-S/8. In Fig. 4, we show that a supervised ViT does not attend well to objects in presence of clutter both qualitatively and quantitatively. We report the Jaccard similarity between the ground truth and segmentation masks obtained by thresholding the self-attention map to keep 60% of the mass. Note that the self-attention maps are smooth and not optimized to produce a mask. Nonetheless, we see a clear difference between the supervised or DINO models with a significant gap in terms of Jaccard similarities. Note that self-supervised convnets also contain information about segmentations but it requires dedicated methods to extract it from their weights [24].

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

Table 6: **Transfer learning by finetuning pretrained models on different datasets.** We report top-1 accuracy. Self-supervised pretraining with DINO transfers better than supervised pretraining.

	Cifar_{10}	Cifar_{100}	INat_{18}	INat_{19}	Flwrs	Cars	INet
ViT-S/16							
Sup. [56]	**99.0**	89.5	70.7	76.6	98.2	92.1	79.9
DINO	**99.0**	**90.5**	**72.0**	**78.2**	**98.5**	**93.0**	**81.5**
ViT-B/16							
Sup. [56]	99.0	90.8	**73.2**	77.7	98.4	92.1	81.8
DINO	**99.1**	**91.7**	72.6	**78.6**	**98.8**	**93.0**	**82.8**

4.2.3 Transfer learning on downstream tasks

In Tab. 6, we evaluate the quality of the features pretrained with DINO on different downstream tasks. We compare with features from the same architectures trained with supervision on ImageNet. We follow the protocol used in Touvron *et al*. [56] and finetune the features on each downstream task. We observe that for ViT architectures, self-supervised pretraining transfers better than features trained with supervision, which is consistent with observations made on convolutional networks [9, 26, 51]. Finally, self-supervised pretraining greatly improves results on ImageNet (+1-2%).

5. Ablation Study of DINO

In this section, we empirically study DINO applied to ViT. The model considered for this entire study is ViT-S. We also refer the reader to Appendix for additional studies.

Importance of the Different Components We show the impact of adding different components from self-supervised learning on ViT trained with our framework.

In Table 7, we report different model variants as we add or remove components. First, we observe that in the absence of momentum, our framework does not work (row 2) and more advanced operations, SK for example, are required to avoid collapse (row 9). However, with momentum, using SK has little impact (row 3). In addtition, comparing rows 3 and 9 highlights the importance of the momentum encoder for performance. Second, in rows 4 and 5, we observe that multi-crop training and the cross-entropy loss in DINO are important components to obtain good features. We also observe that adding a predictor to the student network has little impact (row 6) while it is critical in BYOL to prevent collapse [14, 23]. For completeness, we propose in Appendix an extended version of this ablation study.

Importance of the patch size. In Fig. 5, we compare the k-NN classification performance of ViT-S models trained with different patch sizes, 16×16, 8×8 and 5×5. We also compare to ViT-B with 16×16 and 8×8 patches. All the models are trained for 300 epochs. We observe that the performance greatly improves as we decrease the size of the patch. It is interesting to see that performance can be greatly improved without adding additional parameters. However, the performance gain from using smaller patches comes at the expense of throughput: when using 5×5 patches, the throughput falls to 44 im/s, *vs* 180 im/s for 8×8 patches.

Table 7: **Important component for self-supervised ViT pretraining.** Models are trained for 300 epochs with ViT-S/16. We study the different components that matter for the k-NN and linear ("Lin.") evaluations. For the different variants, we highlight the differences from the default DINO setting. The best combination is the momentum encoder with the multicrop augmentation and the cross-entropy loss. We also report results with BYOL [23], MoCo-v2 [13] and SwAV [9].

	Method	Mom.	SK	MC	Loss	Pred.	k-NN	Lin.
1	DINO	✓	✗	✓	CE	✗	72.8	76.1
2		✗	✗	✓	CE	✗	0.1	0.1
3		✓	✓	✓	CE	✗	72.2	76.0
4		✓	✗	✗	CE	✗	67.9	72.5
5		✓	✗	✓	MSE	✗	52.6	62.4
6		✓	✗	✓	CE	✓	71.8	75.6
7	BYOL	✓	✗	✗	MSE	✓	66.6	71.4
8	MoCov2	✓	✗	✗	INCE	✗	62.0	71.6
9	SwAV	✗	✓	✓	CE	✗	64.7	71.8

SK: Sinkhorn-Knopp, MC: Multi-Crop, Pred.: Predictor
CE: Cross-Entropy, MSE: Mean Square Error, INCE: InfoNCE

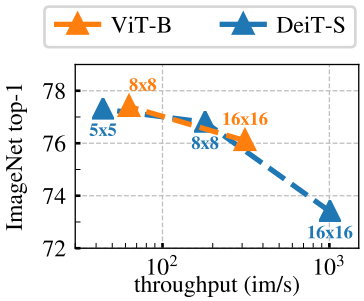


Figure 5: **Effect of Patch Size.** k-NN evaluation as a function of the throughputs for different input patch sizes with ViT-B and ViT-S. Models are trained for 300 epochs.

6. Conclusion

We have shown the potential of self-supervised pretraining a standard ViT model, achieving performance that are comparable with the best convnets specifically designed for this setting. We have also seen emerged two properties that can be leveraged in future applications: the quality of the features in k-NN classification has a potential for image retrieval. The presence of information about the scene layout in the features can also benefit weakly supervised image segmentation.

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

References

[1] Rohan Anil, Gabriel Pereyra, Alexandre Passos, Robert Ormandi, George E Dahl, and Geoffrey E Hinton. Large scale distributed neural network training through online distillation. *arXiv preprint arXiv:1804.03235*, 2018. 3

[2] Yuki Markus Asano, Christian Rupprecht, and Andrea Vedaldi. Self-labelling via simultaneous clustering and representation learning. In *ICLR*, 2020. 2

[3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *preprint arXiv:1409.0473*, 2014. 5

[4] Maxim Berman, Hervé Jégou, Vedaldi Andrea, Iasonas Kokkinos, and Matthijs Douze. MultiGrain: a unified image embedding for classes and instances. *arXiv preprint arXiv:1902.05509*, 2019. 6

[5] Piotr Bojanowski and Armand Joulin. Unsupervised learning by predicting noise. In *ICML*, 2017. 2

[6] Cristian Buciluǎ, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In *SIGKDD*, 2006. 3

[7] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In *ECCV*, 2018. 2, 4

[8] Mathilde Caron, Piotr Bojanowski, Julien Mairal, and Armand Joulin. Unsupervised pre-training of image features on non-curated data. In *ICCV*, 2019. 2

[9] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. Unsupervised learning of visual features by contrasting cluster assignments. In *NeurIPS*, 2020. 1, 2, 3, 4, 5, 8

[10] Mia Xu Chen, Orhan Firat, Ankur Bapna, Melvin Johnson, Wolfgang Macherey, George Foster, Llion Jones, Niki Parmar, Mike Schuster, Zhifeng Chen, et al. The best of both worlds: Combining recent advances in neural machine translation. *preprint arXiv:1804.09849*, 2018. 5

[11] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. *preprint arXiv:2002.05709*, 2020. 2, 3, 5

[12] Ting Chen, Simon Kornblith, Kevin Swersky, Mohammad Norouzi, and Geoffrey Hinton. Big self-supervised models are strong semi-supervised learners. In *NeurIPS*, 2020. 3, 5, 6

[13] Xinlei Chen, Haoqi Fan, Ross Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. *preprint arXiv:2003.04297*, 2020. 5, 8

[14] Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. *preprint arXiv:2011.10566*, 2020. 2, 3, 4, 8

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *preprint arXiv:1810.04805*, 2018. 1, 4, 5

[16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *preprint arXiv:2010.11929*, 2020. 1, 4, 5

[17] Alexey Dosovitskiy, Philipp Fischer, Jost Tobias Springenberg, Martin Riedmiller, and Thomas Brox. Discriminative unsupervised feature learning with exemplar convolutional neural networks. *TPAMI*, 2016. 2

[18] Matthijs Douze, Hervé Jégou, Harsimrat Sandhawalia, Laurent Amsaleg, and Cordelia Schmid. Evaluation of gist descriptors for web-scale image search. In *CIVR*, 2009. 6

[19] Aleksandr Ermolov, Aliaksandr Siarohin, Enver Sangineto, and Nicu Sebe. Whitening for self-supervised representation learning. *preprint arXiv:2007.06346*, 2020. 2

[20] Zhiyuan Fang, Jianfeng Wang, Lijuan Wang, Lei Zhang, Yezhou Yang, and Zicheng Liu. Seed: Self-supervised distillation for visual representation. 2021. 3

[21] Spyros Gidaris, Andrei Bursuc, Gilles Puy, Nikos Komodakis, Matthieu Cord, and Patrick Pérez. Online bag-of-visual-words generation for unsupervised representation learning. *arXiv preprint arXiv:2012.11552*, 2020. 2, 5

[22] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *preprint arXiv:1706.02677*, 2017. 5

[23] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *NeurIPS*, 2020. 2, 3, 4, 5, 8

[24] Shir Gur, Ameen Ali, and Lior Wolf. Visualization of supervised and self-supervised neural networks via attribution guided factorization. *preprint arXiv:2012.02166*, 2020. 7

[25] Michael Gutmann and Aapo Hyvärinen. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *International Conference on Artificial Intelligence and Statistics*, 2010. 2

[26] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *CVPR*, 2020. 1, 2, 3, 4, 5, 8

[27] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 4, 5

[28] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *preprint arXiv:1503.02531*, 2015. 2, 3

[29] Jiabo Huang, Qi Dong, Shaogang Gong, and Xiatian Zhu. Unsupervised deep learning by neighbourhood discovery. In *ICML*, 2019. 2

[30] Allan Jabri, Andrew Owens, and Alexei A Efros. Space-time correspondence as a contrastive random walk. 2020. 7

[31] Sébastien Jean, Kyunghyun Cho, Roland Memisevic, and Yoshua Bengio. On using very large target vocabulary for neural machine translation. *preprint arXiv:1412.2007*, 2014. 4

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

[32] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander M Rush. Opennmt: Open-source toolkit for neural machine translation. *preprint arXiv:1701.02810*, 2017. 5

[33] Zihang Lai, Erika Lu, and Weidi Xie. Mast: A memory-augmented self-supervised tracker. In *CVPR*, 2020. 7

[34] Dong-Hyun Lee et al. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *Workshop on challenges in representation learning, ICML*, 2013. 3

[35] Junnan Li, Pan Zhou, Caiming Xiong, and Steven C.H. Hoi. Prototypical contrastive learning of unsupervised representations. *ICLR*, 2021. 2

[36] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *preprint arXiv:1608.03983*, 2016. 5

[37] Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. 2018. 5

[38] Mehdi Noroozi, Ananth Vinjimoor, Paolo Favaro, and Hamed Pirsiavash. Boosting self-supervised learning via knowledge transfer. In *CVPR*, 2018. 3

[39] Seoung Wug Oh, Joon-Young Lee, Ning Xu, and Seon Joo Kim. Video object segmentation using space-time memory networks. In *ICCV*, 2019. 7

[40] James Philbin, Ondrej Chum, Michael Isard, Josef Sivic, and Andrew Zisserman. Lost in quantization: Improving particular object retrieval in large scale image databases. In *CVPR*, 2008. 6

[41] Boris T Polyak and Anatoli B Juditsky. Acceleration of stochastic approximation by averaging. *SIAM journal on control and optimization*, 30(4):838–855, 1992. 4

[42] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alex Sorkine-Hornung, and Luc Van Gool. The 2017 davis challenge on video object segmentation. *preprint arXiv:1704.00675*, 2017. 7

[43] Filip Radenović, Ahmet Iscen, Giorgos Tolias, Yannis Avrithis, and Ondřej Chum. Revisiting oxford and paris: Large-scale image retrieval benchmarking. 2018. 6

[44] Filip Radenović, Giorgos Tolias, and Ondřej Chum. Fine-tuning cnn image retrieval with no human annotation. *IEEE transactions on pattern analysis and machine intelligence*, 2018. 6

[45] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 1

[46] Jerome Revaud, Jon Almazán, Rafael S Rezende, and Cesar Roberto de Souza. Learning with average precision: Training image retrieval with a listwise loss. In *ICCV*, 2019. 6

[47] Pierre H Richemond, Jean-Bastien Grill, Florent Altché, Corentin Tallec, Florian Strub, Andrew Brock, Samuel Smith, Soham De, Razvan Pascanu, Bilal Piot, et al. Byol works even without batch statistics. *preprint arXiv:2010.10241*, 2020. 2, 4

[48] David Ruppert. Efficient estimations from a slowly convergent robbins-monro process. Technical report, 1988. 4

[49] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge. *IJCV*, 2015. 1, 5

[50] Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *NeurIPS*, 2016. 4

[51] Mert Bulent Sariyildiz, Yannis Kalantidis, Diane Larlus, and Karteek Alahari. Concept generalization in visual representation learning. *arXiv preprint arXiv:2012.05649*, 2020. 8

[52] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. *preprint arXiv:1703.01780*, 2017. 3, 4

[53] Bart Thomee, David A Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. Yfcc100m: The new data in multimedia research. *arXiv preprint arXiv:1503.01817*, 2015. 6

[54] Yonglong Tian, Chen Sun, Ben Poole, Dilip Krishnan, Cordelia Schmid, and Phillip Isola. What makes for good views for contrastive learning. *NeurIPS*, 2020. 5

[55] Giorgos Tolias, Ronan Sicre, and Hervé Jégou. Particular object retrieval with integral max-pooling of cnn activations. *arXiv preprint arXiv:1511.05879*, 2015. 6

[56] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. *preprint arXiv:2012.12877*, 2020. 1, 4, 5, 6, 8

[57] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. 1, 4

[58] Xiaolong Wang, Allan Jabri, and Alexei A Efros. Learning correspondence from the cycle-consistency of time. In *CVPR*, 2019. 7

[59] Tobias Weyand, Andre Araujo, Bingyi Cao, and Jack Sim. Google landmarks dataset v2-a large-scale benchmark for instance-level recognition and retrieval. 2020. 6

[60] Zhirong Wu, Yuanjun Xiong, Stella X Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In *CVPR*, 2018. 2, 4, 5

[61] Junyuan Xie, Ross Girshick, and Ali Farhadi. Unsupervised deep embedding for clustering analysis. In *ICML*, 2016. 2

[62] Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. Self-training with noisy student improves imagenet classification. In *CVPR*, 2020. 3

[63] Qiantong Xu, Tatiana Likhomanenko, Jacob Kahn, Awni Hannun, Gabriel Synnaeve, and Ronan Collobert. Iterative pseudo-labeling for speech recognition. *preprint arXiv:2005.09267*, 2020. 3

[64] I Zeki Yalniz, Hervé Jégou, Kan Chen, Manohar Paluri, and Dhruv Mahajan. Billion-scale semi-supervised learning for image classification. *preprint arXiv:1905.00546*, 2019. 3

[65] Jianwei Yang, Devi Parikh, and Dhruv Batra. Joint unsupervised learning of deep representations and image clusters. In *CVPR*, 2016. 2

[66] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction. *arXiv preprint arXiv:2103.03230*, 2021. 2, 5

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.

[67] Richard Zhang, Phillip Isola, and Alexei A Efros. Colorful image colorization. In *ECCV*, 2016. 5

[68] Hengshuang Zhao, Jiaya Jia, and Vladlen Koltun. Exploring self-attention for image recognition. In *CVPR*, 2020. 1

[69] Chengxu Zhuang, Alex Lin Zhai, and Daniel Yamins. Local aggregation for unsupervised learning of visual embeddings. In *ICCV*, 2019. 2

Authorized licensed use limited to: University of Johannesburg. Downloaded on August 15,2026 at 00:37:50 UTC from IEEE Xplore. Restrictions apply.